6.1 Introduction

This document presents the architecture and detailed design for Gather, our senior project. Gather is a web application that collects a student's assignments, exams and deadlines in one place. Tasks can be entered by hand or imported from Brightspace, PDFs (such as a syllabus) and course web pages. Anything imported from an outside source must be reviewed and approved by the student before it appears in their task list and calendar.

Gather is under active development. Each section marks every item as **Implemented** or **Planned** so the reader can tell what exists in the code today and what is still design only.

6.1.1 System Objectives

Students currently check Brightspace, each syllabus and sometimes a professor's own website to learn what is due, and deadlines are easy to miss. The objective of Gather is to give each student a single, trusted view of everything that is due, without letting unreviewed data from outside sources change that view. Specifically, the application shall:

- Show one dashboard and calendar listing all assignments by due date, and let the student mark them complete (SRS 5.3.1).
- Let students add, edit and delete tasks manually and save them to their own account (SRS 5.3.1, 5.3.2).
- Connect to Brightspace with the student's permission and retrieve course deadlines, and accept PDFs and web page links as additional sources (SRS 5.3.2).
- Use Llama 3 through OpenRouter to find assignment information in imported text, storing results as proposals that become tasks only after student approval (SRS 5.2.1, 5.3.2).
- Protect user data: each user can access only their own records, Brightspace tokens are encrypted, and every Brightspace API call is logged (SRS 5.3.3).
- Meet response targets: dashboard load under 5 seconds, task updates under 3 seconds, and imported items visible for review within 10 seconds after processing completes (SRS 5.4).

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 Graphical User Interface

Students use Gather through a web browser. The interface is a single-page React application with a fixed left sidebar containing four navigation items: Dashboard, My Tasks, Review Imports and Sources. It supports mouse or trackpad, touch, and keyboard input.

Implemented (prototype, sample data): The Dashboard page shows summary counts (total, to do, completed), an Add Task button, the assignment list with completion checkboxes, a pending-imports panel with Approve and Reject buttons, and a slide-out calendar (Figure 6-1).

Planned: Sign in, sign up, password reset, settings, PDF/URL upload and source management pages (SRS 5.3.1). Review Imports will list each proposed assignment with its title, course, due date and the source it came from, with Approve, Edit-then-approve and Reject actions. Sources will list the connected Brightspace account and past imports, with Connect and Disconnect actions.

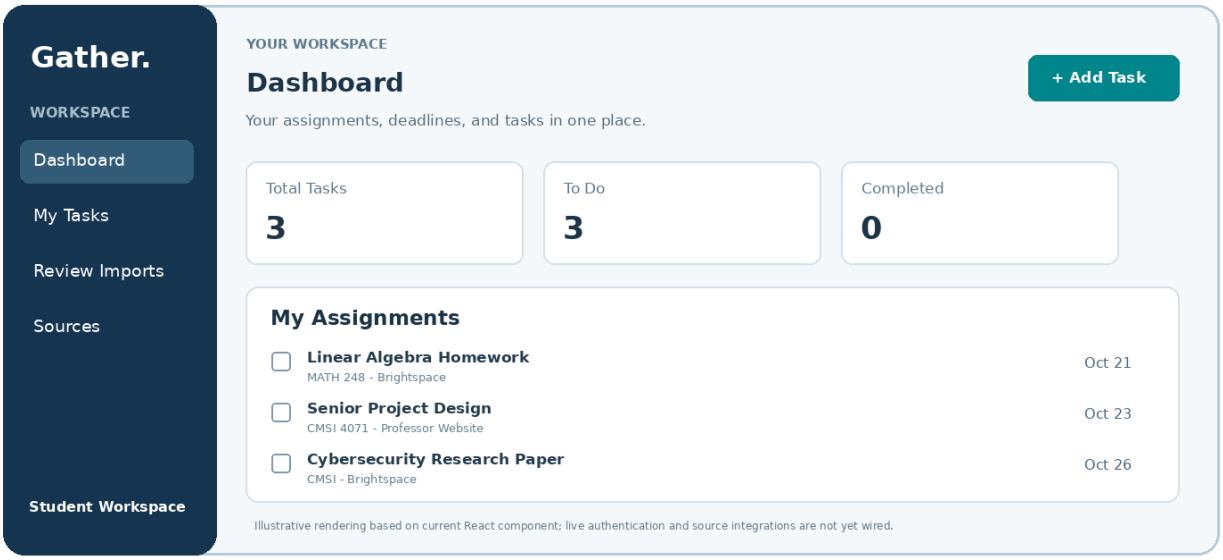


Figure 6-1. Mockup of the Gather dashboard, based on the current React layout (not yet connected to the backend).

6.1.2.2 Display Hardware

Gather requires a screen on a laptop, desktop, tablet or phone. The layout targets widths from a phone (about 375 px) up to desktop monitors. No special display hardware is required.

6.1.2.3 Keyboard, Pointing and Touch Input

All actions can be performed with a mouse or trackpad, a touchscreen, or a keyboard (Tab to move between controls, Enter or Space to activate them). Gather uses only standard browser input events and needs no device drivers.

6.1.2.4 Network Hardware

The student's device needs an internet connection (Wi-Fi, Ethernet or cellular). All server-side code runs on Firebase Cloud Functions, so the team operates no server hardware. Development is done on Windows, macOS and Linux machines.

6.1.2.5 Network and Communication Protocols

All traffic between the browser and the backend uses HTTPS (HTTP over TLS 1.2 or higher). Requests and responses use JSON (`application/json`). Routes that require a signed-in user carry the Firebase ID token in an `Authorization: Bearer <token>` header. PDF uploads use `multipart/form-data` (Planned); URL imports are sent as JSON. When the backend exchanges OAuth codes for tokens with Brightspace, it sends `application/x-www-form-urlencoded` bodies, as Brightspace requires.

Status: `index.js` currently registers only `express.json()`. Multipart upload handling (6.1.2.15) and the CORS policy (6.1.2.12) are Planned.

6.1.2.6 React

Version: 19.2.8 (react and react-dom). **Purpose:** builds the entire user interface. **Interfaces with:** CSC-1 Web Client, in `frontend/src/App.jsx`. **Status:** Implemented as a prototype; the task list is held in React state (`useState`) with sample data and is not yet loaded from the API.

6.1.2.7 Vite

Version: 8.3.0, with @vitejs/plugin-react 6.1.1. **Purpose:** local development server and production build of the frontend. **Interfaces with:** CSC-1 only; it has no role at runtime. **Status:** Implemented.

6.1.2.8 Node.js and Express

Version: Node.js 24 (set in `Backend/functions/package.json`) and Express 4.21.2. **Purpose:** HTTP routing and middleware for the API. **Interfaces with:** CSC-2 and CSC-3. **Status:** Implemented with three route groups: `/assignments`, `/users` and `/brightspace`.

6.1.2.9 Firebase Cloud Functions

Version: firebase-functions 7.0.0. **Purpose:** deploys and runs the Express application. The whole API is exported as one HTTPS function named `api`. **Interfaces with:** CSC-2 through CSC-5. **Status:** Implemented.

6.1.2.10 Firebase Admin SDK, Authentication, Firestore and Cloud Storage

Version: firebase-admin 13.6.0. **Purpose:** server-side verification of Firebase ID tokens, reading and writing Cloud Firestore, and (Planned) storing temporary PDF uploads in Cloud Storage. Firestore holds user profiles, assignments, Brightspace connection data and Brightspace usage logs. **Interfaces with:** CSC-2 (token checks), CSC-3, CSC-4 and CSC-6. **Status:** Token verification and Firestore access Implemented; Cloud Storage Planned.

6.1.2.11 Firebase Authentication Client SDK

Version: firebase (JavaScript SDK) [TO CONFIRM: version to be added to frontend package.json]. **Purpose:** lets the student sign in from the browser and obtain the ID token sent to the API. **Sign-in providers:** [TO CONFIRM: e.g. email/password and Google]. **Interfaces with:** CSC-1. **Status:** Planned.

6.1.2.12 cors

Version: 2.8.5. **Purpose:** restrict API requests to Gather's own frontend origin. **Interfaces with:** CSC-2. **Status:** Listed in package.json; not yet registered in index.js (Planned).

6.1.2.13 Brightspace (D2L) API

Version: Brightspace LE API [TO CONFIRM: version supported by LMU Brightspace, from /d2l/api/versions/]. **Protocol:** OAuth 2.0 Authorization Code flow (RFC 6749) with PKCE using the S256 method (RFC 7636). LMU ITS requires PKCE for every application that connects to Brightspace. The server builds the authorize, token and revoke URLs from BRIGHTSPACE_BASE_URL and also reads BRIGHTSPACE_CLIENT_ID, BRIGHTSPACE_CLIENT_SECRET, BRIGHTSPACE_REDIRECT_URI and BRIGHTSPACE_SCOPES from the environment. **Interfaces with:** CSC-3. **Status:** connect, callback, status and disconnect endpoints Implemented; course and assignment retrieval Planned.

LMU ITS requires a record of every use of the Brightspace API. All Brightspace calls (token exchange, refresh, revoke, and later course and assignment retrieval) therefore pass through a single wrapper function that writes a usage log entry (see 6.2.2 and 6.3.1.10, 6.3.3.5).

6.1.2.14 OpenRouter / Llama 3

Version: OpenRouter chat completions API; model [TO CONFIRM: final choice between meta-llama/llama-3.1-8b-instruct and meta-llama/llama-3.3-70b-instruct after testing]. **Purpose:** extract assignment titles, courses and due dates from cleaned source text. The backend sends the text over HTTPS with an API key held only on the server and requests JSON output, which is validated against a schema. Rate-limit responses are retried with backoff. **Interfaces with:** CSC-5. **Status:** Planned.

6.1.2.15 PDF Parsing and Web Page Fetching

Libraries: pdf-parse for PDF text [TO CONFIRM: version]; cheerio for HTML text extraction [TO CONFIRM: version]; busboy for multipart uploads in Cloud Functions [TO CONFIRM: version]. **Purpose:** accept PDFs up to 5 MB and http/https links, extract their text, and pass it to CSC-5. Uploaded PDFs are stored temporarily in Cloud Storage and deleted once text is extracted (SRS). **Interfaces with:** CSC-4. **Status:** Planned.

6.1.2.16 Node.js crypto (and @google-cloud/kms)

Version: crypto module built into Node.js 24. **Purpose:** generates the PKCE verifier and challenge, and encrypts Brightspace refresh tokens with AES-256-GCM using a key from the TOKEN_ENCRYPTION_KEY environment variable. **Interfaces with:** CSC-3. **Status:** Implemented. @google-cloud/kms 6.3.0 is listed in package.json for possible future key management but is not called by any code. Content from Brightspace and professor websites is always treated as untrusted data.

6.2 Architectural Design

Gather is divided into six Computer Software Components (CSCs) by responsibility: the web client, the API and identity layer, the Brightspace integration, data ingestion, AI extraction and review, and data management. Each CSC owns one kind of external dependency, so PDF parsing, Brightspace and the LLM never share code and can be built and tested by different team members.

Two interfaces keep coupling low. First, every database access goes through a user-scoped repository interface that only reads and writes under `users/{uid}`, so no handler can reach another user's data. Second, ingestion and extraction are defined as interfaces (TextSource and AssignmentExtractor). Adding a new source type or swapping the LLM changes one class and does not affect the review flow. The diagrams in this section are high level; details are in 6.3.

6.2.1 Major Software Components

Component	Main classes	SRS coverage	Status
CSC-1 Web Client	React pages and components	5.3.1	Prototype; login and API calls Planned
CSC-2 API / Identity	AuthMiddleware, AssignmentRouter, UserRouter, ImportRouter	5.3.2.1 to 5.3.2.11	Main routes Implemented
CSC-3 Brightspace Integration	BrightspaceRouter, BrightspaceClient, CryptoHelper	5.3.2.1 to 5.3.2.11	OAuth Implemented; sync Planned
CSC-4 Data Ingestion	TextSource, PdfSource, UrlSource	5.3.2.12 to 5.3.2.18	Planned
CSC-5 AI Extraction and Review	AssignmentExtractor, LlamaExtractor, ReviewService	5.3.2.12 to 5.3.2.18	Planned (UI demo only)
CSC-6 Data Management	UserScopedRepository and Firestore repositories	5.3.3	Partly Implemented

Table 6-1. Major software components and their SRS functional requirements.

CSC-1 Web Client. Runs in the browser. Renders the dashboard, task list, calendar, manual task form, import review screen and source management, and sorts and filters tasks. It holds no business rules beyond input checks; all saved data goes through CSC-2.

CSC-2 API / Identity. The single entry point to the backend. AuthMiddleware verifies the Firebase ID token on every protected route and attaches the user id to the request. The routers validate request bodies and call CSC-3, CSC-4, CSC-5 or CSC-6.

CSC-3 Brightspace Integration. Runs the PKCE OAuth flow required by LMU ITS, encrypts and stores the refresh token, disconnects (revoke and delete), and will retrieve courses and assignments. BrightspaceClient is the only class that calls Brightspace, which is how every call is guaranteed to be logged.

CSC-4 Data Ingestion. Accepts a PDF or URL, extracts and cleans the text, records where it came from, and manages temporary files in Cloud Storage.

CSC-5 AI Extraction and Review. Sends cleaned text to Llama 3, validates and normalizes the returned JSON, saves results as candidates, checks for duplicates, and converts approved candidates into assignments.

CSC-6 Data Management. Stores user profiles, assignments, encrypted Brightspace tokens and usage logs in Firestore, and later import jobs, candidates and courses. All access is scoped to the signed-in user.

6.2.2 Major Software Interactions

Sign in (CSC-1 to Firebase Authentication). The browser signs in through the Firebase Auth client SDK and receives a short-lived ID token, which the SDK refreshes automatically. Status: Planned.

Browser to API (CSC-1 to CSC-2). Each user action sends an HTTPS request with a JSON body and the ID token. AuthMiddleware verifies the token through the Admin SDK before any handler runs; an invalid or missing token ends the request with 401. Status: designed; the prototype does not call the API yet.

API to Firestore (CSC-2 to CSC-6). Handlers call repository functions that use the Admin SDK. All data is stored under `users/{uid}`, which keeps each user's records separate.

Brightspace connection (CSC-2 to CSC-3 to Brightspace). The student is redirected to Brightspace to grant access. Brightspace returns to the callback with a code and a state value. The server confirms the state matches the saved value, exchanges the code (with the PKCE verifier) for tokens over HTTPS, and stores the refresh token encrypted in Firestore. Disconnecting revokes the token where possible and deletes the stored connection data.

Brightspace usage logging (CSC-3 to CSC-6). Because LMU ITS requires a record of every Brightspace call, BrightspaceClient writes the log entry *before* the call, with status "pending", the user, and the operation. If that write fails, the Brightspace call is not made and the user sees an error, so no call can happen without a record. After the call, the entry is updated to "success" or "failed". If only this second update fails, the error goes to Cloud Functions logs and the pending entry remains as the record.

Import to Cloud Storage and AI (CSC-4 to CSC-5). A PDF is written to Cloud Storage, its text is extracted, and the file is deleted. A URL is fetched over HTTPS. The cleaned text, with its source, is passed to the AssignmentExtractor, which calls OpenRouter and returns proposed assignments. These are saved as candidates and never become tasks automatically. Status: Planned.

Review to database (CSC-1 to CSC-5 to CSC-6). When the student approves a candidate, ReviewService checks for a duplicate and then creates a normal assignment. A rejected candidate is marked rejected and kept so the same item is not proposed again.

Errors. If a source fetch, PDF parse or AI call fails, the import job is marked failed, the student sees an error, and existing tasks are not changed.

6.2.3 Architectural Design Diagrams

Three diagrams follow: a use case diagram (Figure 6-2), a top-level class diagram with the main classes and interfaces (Figure 6-3), and a component/deployment diagram (Figure 6-4). Elements shaded light yellow, or marked with an asterisk, are Planned. More detailed diagrams are in 6.3.4.

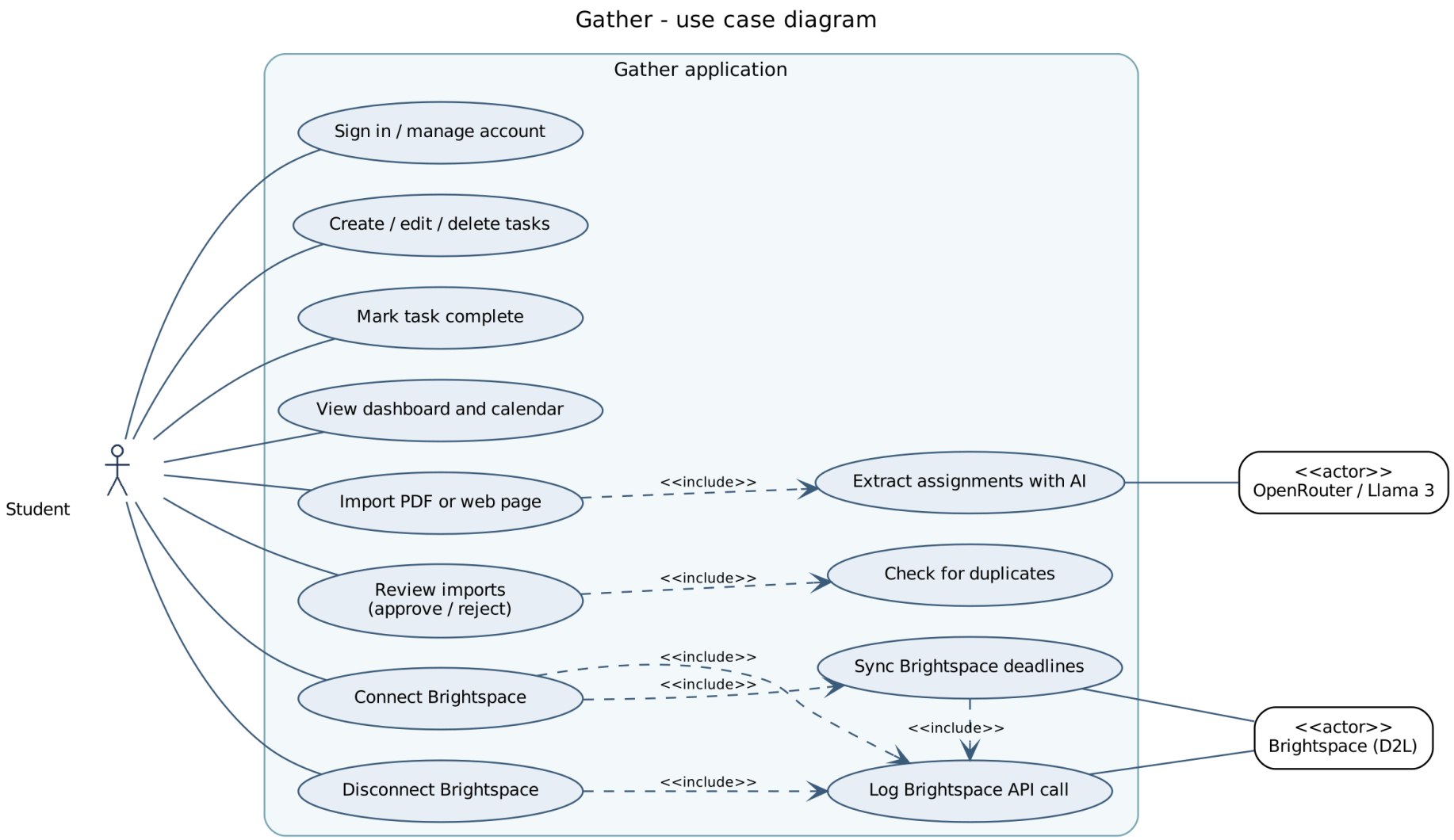


Figure 6-2. Use case diagram.

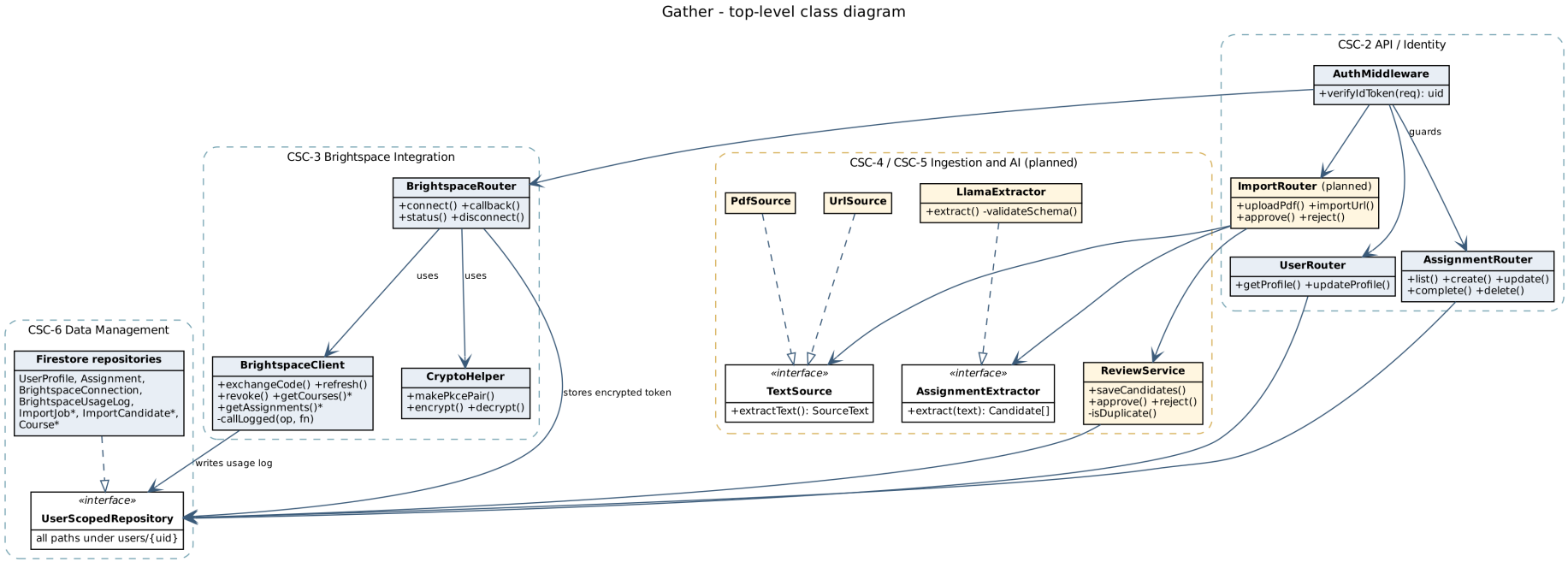


Figure 6-3. Top-level class diagram (* = Planned method or collection).

Gather - component / deployment diagram

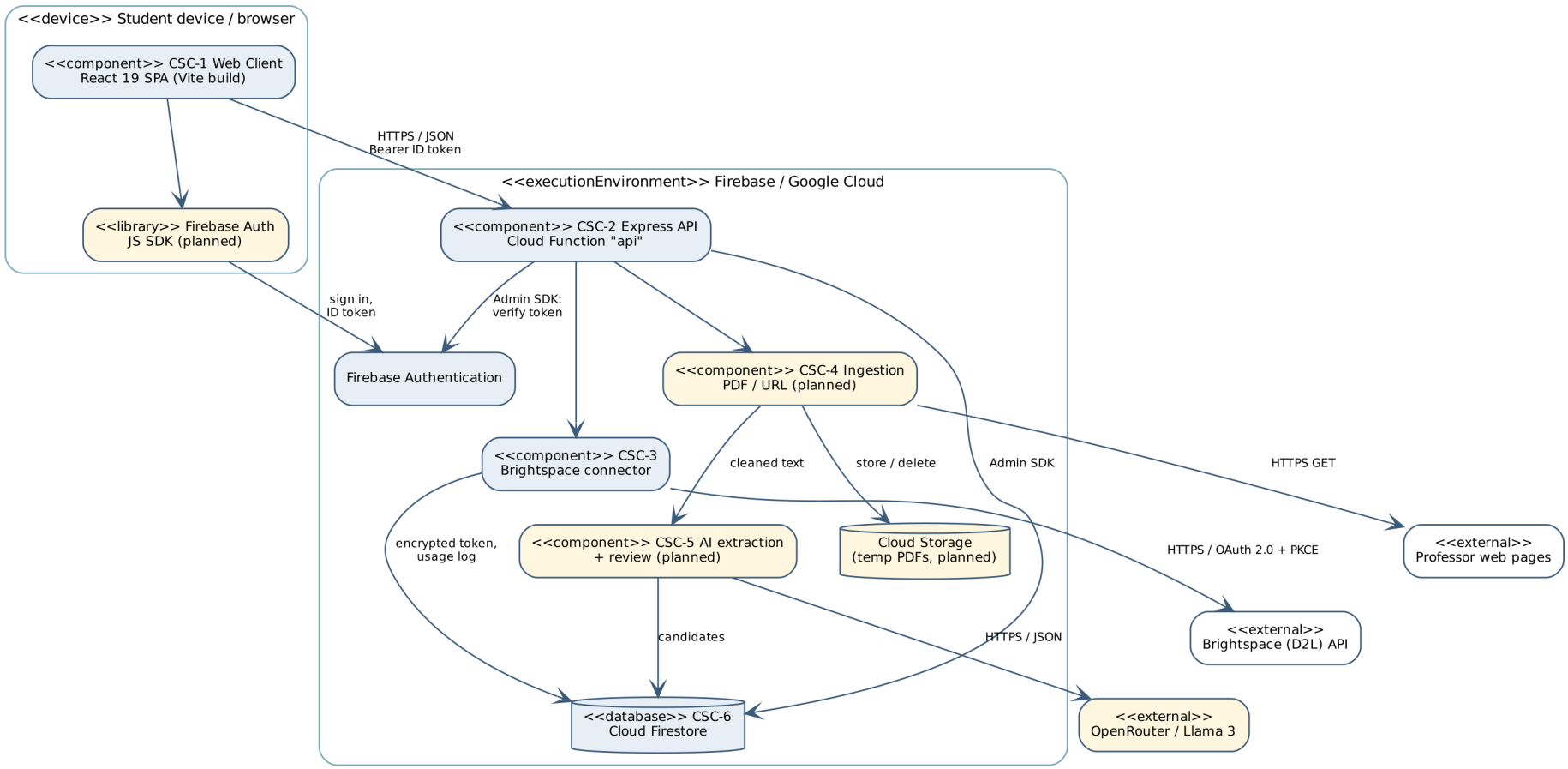


Figure 6-4. Component / deployment diagram.